

Agentic Fuzzing Report: Mini-XML (mxml)

Design Overview

This project implements an autonomous, agentic fuzzing loop designed to discover C-level memory corruption vulnerabilities in the Mini-XML (`mxml`) library. Unlike traditional fuzzers that rely on static mutation or coverage-guided feedback (like AFL), this system utilizes a Large Language Model (LLM) to iteratively evolve Hypothesis-based Python strategies.

System Architecture

The architecture consists of four primary components:

- **Orchestrator:** A control loop that manages the fuzzing iterations. It triggers the strategy generation, executes the fuzzing harness, and handles the feedback loop for refinement.
- **Agentic Generator:** A system that uses an LLM to author complex Python modules using the `hypothesis` library. It uses structured prompts to guide the LLM toward generating high-entropy, malformed XML.
- **Fuzzing Harness:** A C-based driver that wraps the `mxml` library. It is compiled with AddressSanitizer (ASan) and UndefinedBehaviorSanitizer (UBSan) to detect memory leaks, buffer overflows, and undefined behavior.
- **Triage Engine:** An automated post-processing unit that deduplicates crashes by signature, minimizes crashing inputs to the smallest possible reproduction case, and generates reports.
-

Strategy Steering & Feedback

Since `mxml` does not provide code coverage instrumentation (like edge coverage in AFL), the refinement process is steered by a **proxy signal**: the **Acceptance Rate** and **Crash Discovery**.

- **Acceptance Rate:** Measures the ratio of inputs that `mxml` considers "valid" vs. "malformed." A very high acceptance rate suggests the strategy is too "safe" (generating well-formed XML), while a very low rate indicates it is producing syntactically impossible noise.
- **Crash Discovery:** The presence of sanitizer violations (ASan/UBSan) or unexpected SIGSEGV/SIGABRT signals.

When a refinement cycle begins, the LLM is provided with the previous strategy code and the execution summary. If zero crashes were found, the LLM is instructed to switch to "Aggressive Combination" mode, combining multiple malformation vectors (e.g., massive attributes + deep nesting + null bytes) to break parser invariants.

Grammar & Adaptation

The fuzzing was guided by the `mxml` ANTLR-derived grammar. To increase the effectiveness of the fuzzer, the generator was explicitly instructed to target known "weak" parsing paths:

- **Mismatched Tags:** Exploiting stack/tree management during tag closure.
- **Duplicate Attributes:** Testing the robustness of attribute mapping and memory allocation for attribute lists.
- **Entity Reference Abuse:** Stressing the string allocation and lookup logic for undefined entities.
- **Large Payloads:** Targeting heap-based buffer overflows via extremely long attribute values and text content.
-

Findings

Execution Environment

The fuzzing loop was executed on a Kaggle environment. Initially, attempts were made using the Groq API, but rate-limiting constraints necessitated a transition to running an open-source model locally. Specifically, **Qwen/Qwen2.5-7B-Instruct** was utilized as the LLM backend via a custom HuggingFace Transformers client.

Results Summary

Metric	Value
Total Examples Generated	2,500
Total Iterations	5
Crash Candidates Found	319
Unique Crash Signatures	4

Crash Analysis

The fuzzer successfully identified 4 distinct crash signatures during the 5-iteration loop. While 319 crash events were detected, triage reduced these to 4 unique bug classes. All crashes were **LeakSanitizer** violations (ASan code 3) — `mxml` failed to free memory on error-exit paths, causing the sanitizer to flag the process at exit.

#	Signature	Leak Size	Trigger	Minimized Reproducer	Source File
1	1d30d454	560 B (5 allocs)	Attribute name with high-byte chars + null-padded binary value	<HF_MODEL_NAME x="<64 KB binary blob with \x00...">	mxml-node.c:931 / mxml-attr.c:255
2	45bedb75	*(nil)*	Mismatched nested tags: <V7><addresssanitizer></V7></addresssanitizer>	<V7><addresssanitizer></V7></addresssanitizer> (46 B)	*(no leak — parse-path abort)*
3	50e35c86	94 B (2 allocs)	Unquoted attribute with control chars (\x1f\x83\xdb...) + unterminated entity	<HZRtu x="<52 KB binary blob with control chars...">	mxml-node.c:931 / mxml-private.c:99
4	bff45faa	20 352 B (5 allocs)	Duplicate attribute x with 60 KB null-byte string on digit-prefixed tag <0>	<0 x="<60 KB null bytes">	mxml-node.c:931 / mxml-attr.c:255 / mxml-private.c:99

Key Observations

- **3 of 4 crashes are LeakSanitizer violations** — mxml allocates a node and/or attribute string on the hot path, encounters a well-formedness error (duplicate attr, mismatched tag, invalid name), enters an error path, and returns `NULL` without freeing the partially-allocated structures.
- **Crash 2 is a parse-abort without memory leak**, triggered by the simplest reproducible input (46 bytes). It confirms the mismatched-tag error path is also under-tested.
- **Crashes 1, 3, 4 share the same root cause:** `mxml_new()` allocates the node, then `mxml_set_attr()` allocates the attribute copy, but neither path cleans up when a subsequent error (duplicate attribute, invalid name, bad entity) causes an early return.

Strategy Evolution

The strategy evolved from generating mostly well-formed XML (high acceptance, zero crashes) to generating highly corrupted, "hostile" XML. By iteration 3, the LLM had learned to combine massive attributes with unterminated comments and deep nesting, which directly resulted in the discovery of the first sanitizer violations.

Challenges

LLM Reliability & Prompt Engineering

The most significant challenge was the "unreliability" of the LLM output. Even with strict system prompts, the model frequently exhibited several failure modes:

- **Code-Prose Mixing:** The LLM often included conversational English (e.g., "Here is your strategy...") within the response, which broke the Python parser. This required a robust `_extract_python` utility using regex and strict prompt enforcement.
- **Structural Hallucinations:** The model often defined the `xml_strategy` inside a helper function rather than at the module level, causing AST validation failures. I addressed this by providing an explicit, mandatory structural template in the `refine_prompt.md`.
- **Broken Logic:** The LLM frequently hallucinated Hypothesis APIs or used `st.recursive` incorrectly (passing a strategy object instead of a `lambda` for the `extend` argument). This necessitated a multi-stage validation pipeline (AST check $\rightarrow$ Compile check $\rightarrow$ Live-load test) to ensure only valid code reached the harness.
-

Resource Constraints

Due to the lack of access to high-performance/unlimited LLM APIs, I had to adapt the pipeline to run on Kaggle. This introduced challenges in managing GPU memory and handling the slower inference speeds of open-source models compared to hosted APIs. The transition to `Qwen2.5-7B-Instruct` required implementing a custom `LLMClient` that could handle HuggingFace-specific loading and device mapping.

Future Work

Given more time and access to coverage-guided feedback (e.g., using `AFL-style` instrumentation), the fuzzer could be significantly more efficient. The current "proxy signal" method is effective but brute-forces many irrelevant paths. Integrating real-time edge coverage would allow the LLM to focus its creative energy on exploring new, unvisited code paths in the C source.

Appendices

Note: Detailed logs, generated strategies, and crash reports are located in the `fuzzer/logs/`, `fuzzer/strategies/`, and `triage/crashes/` directories respectively.